PHY4605 Computational Methods in Physics
Learning Note | Week 13

Computational Investigation: Present, defend, and hand off a validated physical model

Turn one bounded computational investigation into evidence that another physics student can understand, check, and reproduce.

Physical question. How can a group present one changed physical model, one main output, and two checks so that its conclusion is defensible without relying on code volume or a full AI chat history?

Learning outcomes

1. state the approved physical model, assumptions, variables, parameters, and units;
2. trace the starter Live Script, justify one controlled modification, and connect it to one main graph or table; and
3. defend the result with a required check, a genuinely different chosen check, a physical interpretation, and a reproducible handoff.

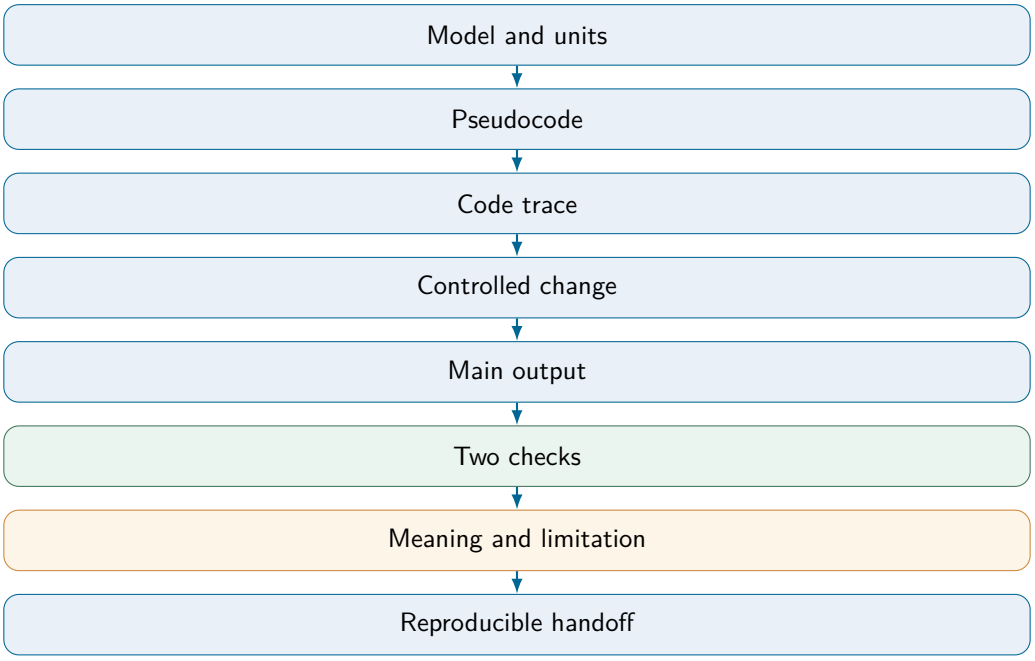
How to use this note

This is a completion and communication week, not a new numerical-method chapter. Start from the problem space, method, and validation plan approved earlier in the course. Use the supplied `Week13_Capstone_Starter_and_Validation.m` file as a runnable starter, then adapt only the bounded section your group is responsible for. Read each prompt before running MATLAB and keep the model, code, output, checks, and explanation together.

The required route. One question; one controlled change; one main output; one required check; one different chosen check; one limitation; one fresh-session handoff. A fresh-session handoff means another student can rerun the edited starter from a clean MATLAB session using the recorded inputs and settings. Independent extensions and deeper method comparisons are optional extensions, not required for ordinary completion.

1. The capstone evidence chain

The final presentation is a short scientific argument. Each part answers a different question, so the audience can follow the reasoning rather than guess it from a polished graph.



Read the sequence from top to bottom. The model and units prevent a silent change of physical meaning. The pseudocode makes the order inspectable. The code trace shows what the script actually computes. The main output answers the one question. The checks support specific claims. Meaning and limitation prevent overclaiming. The reproducible handoff lets another student repeat the sequence.

2. Keep the investigation bounded

The capstone is a supported investigation rather than an open-ended research project. The scope below is the ordinary required route.

Evidence item	**What the group must show**
Physical question	One narrow question about a supplied or approved physical behaviour.
Controlled change	One parameter or assumption changes; all other important inputs are named as fixed.
Main output	One labelled graph or table that directly answers the question, with quantities, units, and cases.
Required check	The validation check agreed in the model and method plan.
Chosen check	A second check using genuinely different evidence or testing a different weakness.
Interpretation	One physical conclusion linked to the prediction.
Limitation	One numerical or model limitation that marks where the claim stops.
Handoff	Edited starter, inputs, settings, run order, clean-session record, and concise decision evidence.

Do not broaden the question at the last moment. A second graph, an extra method, or an independent extension can be interesting, but it is not a substitute for a complete required evidence chain. Remove optional material if it hides the main output or the checks.

3. Model first: variables and units

Before showing MATLAB, state the physical picture, assumptions, and output. Every important code variable should map to a physical quantity and unit. Keep numerical settings visible because a timestep, grid, angle array, or reference value is part of the evidence.

Record	Example name	Purpose
Question	range versus launch angle	identifies the physical output
Parameter	`v0_mps`	changed or fixed input with unit m s^{-1}
Discretisation	`angle_deg = 0:5:90`	finite set of model inputs
Method	parameter sweep	supplied computational recipe
Output	`range_m`	main result with unit m
Check	45 degree reference	known result or independent evidence

The example track in the starter uses level landing without air drag:

$$R(\theta) = \frac{v_0^2 \sin(2\theta)}{g}.$$

In words: horizontal range is launch speed squared, multiplied by the angle factor, divided by the gravitational acceleration. The supplied baseline uses $v_0 = 18\,\mathrm{m\,s^{-1}}$ and the controlled modification uses $v_0 = 20\,\mathrm{m\,s^{-1}}$, while $g = 9.81\,\mathrm{m\,s^{-2}}$ and the angle array remain fixed. The practical brief and starter use the same five canonical supplied tracks. Your approved track may use any one of these familiar models.

Prediction checkpoint. Before running the modified case, state whether the main output should increase, decrease, change sign, approach a bound, or change shape. A prediction gives the final output a testable reference.

4. Pseudocode before code

Use short plain-language steps. A teammate should be able to point to the job of each code block before seeing its MATLAB syntax.

1. record the approved model, assumptions, inputs, and units;
2. calculate the baseline output with the supplied method;
3. change one parameter or assumption and keep the other inputs fixed;
4. calculate the modified output and choose one main graph or table;
5. run the required check and a different chosen check; and
6. interpret the pattern, state one limitation, and record the clean-session handoff.

Code-tracing prompt. Choose one relevant line. What quantity does it represent? Which pseudocode step does it complete? What output would reveal a wrong sign, wrong unit, wrong index, or wrong array operation?

5. A worked validation example

For the supplied projectile track, the 45 degree result has a known form:

$$R(45^\circ) = \frac{v_0^2}{g}.$$

The script compares the calculated baseline value at 45 degrees with this reference. This is the required check for the default track. The chosen check uses a different relationship: the range curve should be symmetric, so $R(\theta) = R(90^\circ - \theta)$ for the chosen angle array. These are two genuinely different validation checks: the reference value tests agreement at one known point, while symmetry tests the shape of the whole modelled relationship. Two versions of the same comparison, such as repeating the endpoint check at two speeds or comparing the same curve at two resolutions, do not count as two different checks.

Listing 1: MATLAB code for the projectile baseline output and two genuinely different validation checks

```
range_m = v0_mps^2*sind(2*angle_deg)/g_mps2;
angle_45_index = find(angle_deg == 45,1);
reference_range_45_m = v0_mps^2/g_mps2;
reference_error_m = abs(range_m(angle_45_index)-reference_range_45_m);
symmetry_error_m = max(abs(range_m-fliplr(range_m)));
assert(reference_error_m < 1e-12)
assert(symmetry_error_m < 1e-12)
```

The first assertion tests agreement with a known value. The second checks a different structural property of the model. Both pass for the supplied baseline. Passing checks do not prove that the no-air-drag assumption is realistic; they support narrower claims about the calculation and the stated model.

Validation language. Say what the evidence supports and what it does not establish. "The code looks reasonable" is not a validation argument; a value, bound, reference, residual, conservation relation, or clean rerun is concrete evidence.

6. Main output and physical meaning

The main graph or table should answer the original question directly. Label the measured or modelled quantity, its unit, the baseline case, the modified case, and the changed parameter. State one pattern for the audience to notice before adding secondary detail. For the projectile example, the modified speed increases the range at the same angle because the model contains v_0^2. The curve peaks near 45 degrees for the level-landing, no-drag model. A limitation is that the ideal model omits air resistance and therefore does not claim to predict a real projectile in every condition. Keep two kinds of limitation separate:

- numerical limitation: resolution, timestep, approximation, or reference comparison; and
- model limitation: an idealisation such as no drag, constant parameters, or expected count rather than an individual random count.

Guided extension

After the required output is clear, a group may use supplied evidence from a second numerical resolution, a second familiar method, or a second parameter range to deepen the discussion. Treat this as supporting evidence, not as a replacement for the one main output and two checks.

Optional extension

An independent extension or deeper method comparison may be added after the complete required handoff. It is optional and not required for ordinary completion; removing it must leave the model, output, validation, interpretation, and reproducibility pathway intact.

7. Defence and responsible AI evidence

The shared group result does not remove the individual explanation. Each member should trace one relevant code section, explain its physical meaning, and answer one parameter-change or validation follow-up. The lecturer keeps a rotation ledger with one entry per student: complete one defence for every group member before assigning a second. Mark an absence as pending and add a make-up entry when the deferred defence is completed.

AI decision record. Write one short record: what assistance was requested; what was accepted, modified, or rejected; why; and which independent checks supported the final claim. Do not submit a complete chat transcript, and do not use an AI-detection score as evidence of learning or misconduct.

8. Reproducible handoff

Another student should be able to start from a fresh MATLAB R2025a-or-later session and regenerate the main output and both checks. The handoff must record:

1. the edited starter Live Script, required input files, parameter values, assumptions, units, and numerical settings;
2. the run order and MATLAB release;
3. the main output plus both checks, numerical evidence, physical conclusion, and one limitation; and
4. the concise AI decision record and individual defence entries, including any pending absence or make-up record.

Run the final script from the top in a fresh session before marking the evidence final in the submission record. Keep the edited script and inputs together. If a dependency is not base MATLAB, name it explicitly rather than leaving another student to guess.

Final evidence state. Mark the evidence *final in the submission record* only after the clean rerun reproduces the main output and both checks, the handoff record is complete, and every group member has a recorded defence entry or a clearly pending make-up entry.

Preparation for the presentation. Open the starter script, select the approved track, read the model and units aloud, make the prediction, trace one code line, and locate both validation results. Bring the editable script and handoff checklist to the group review.

Three takeaways

1. A bounded question makes a computational result easier to interpret and defend.
2. Two checks are useful only when they test different claims or weaknesses.
3. Reproducibility is part of the scientific result: another student should know what to open, run, check, and conclude.